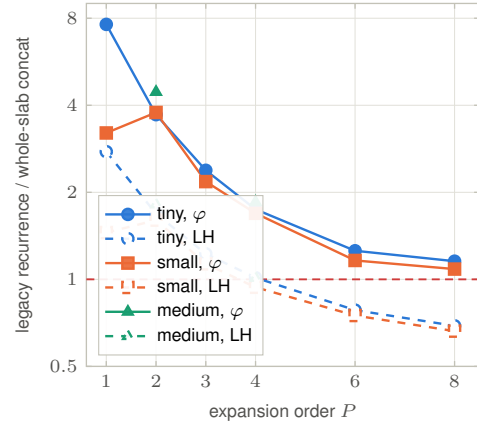
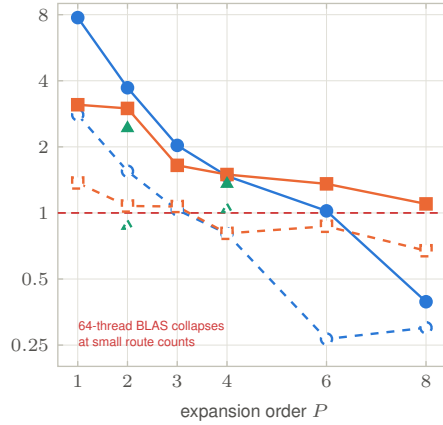


**Fig. 2 — Whole-slab speedups across order, route count, and integrated GPU problem size**

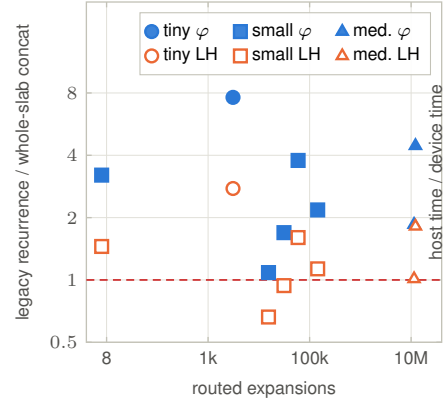
**(a) CPU host stage, BLAS=1**



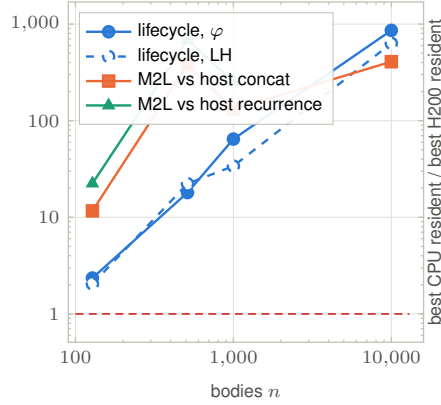
**(b) CPU host stage, BLAS=64**



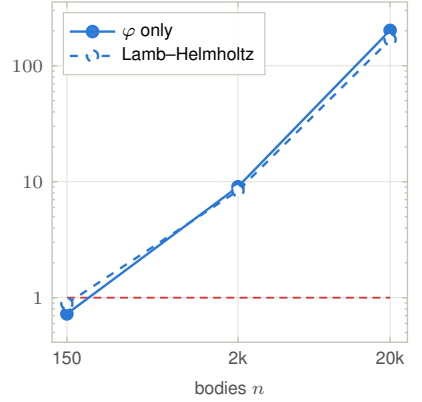
**(c) CPU speedup vs route count, BLAS=1**



**(d) 019b H200 corner vs host,  $P = 4$**



**(e) integrated 024 platform speedup**



**Machines / regimes:** (a–c) m12-1-7, AMD EPYC 7763, OpenBLAS pinned at process start to 1 and 64 threads, Float64. (d) m13h-1-1, NVIDIA H200 (140 GiB, CUDA 12.8), device-resident lifecycle, ConcatenatedFixedZM2L. (e) matched task-024 uniform Float64  $P = 4$  cases: m12-2-17 EPYC 7763 BLAS=1 vs m13h-1-1 H200, fastest eligible resident strategy on each platform. **Data:** task 019b, `crossover_stage_blas{1,64}.csv` and `m13h-1-1/gpu_smallp.csv`; task 024, `operator_ab_benchmark/summary/case_rankings.csv` (tables `fig02a_*`, `fig02b_*`, `fig02c_*`, `fig02d_*`). **Reads as:** unlike the per-column operators of Fig. 1, the whole-slab concat path wins on the CPU at small  $P$  ( $7.6\times$  at  $P = 1$ ,  $1.7\times$  at  $P = 4$ ) but gives the advantage back for Lamb–Helmholtz at  $P \geq 6$ . The explicit route-count panel answers the “vs routes” question directly and its answer is no: route count alone does not predict the win. The largest speedup in the whole sweep ( $7.6\times$ ) is at the *smallest* population, 3096 routes, while 11.4M routes gives  $1.9\times$  and 8 routes — a legitimate tiny-batch extreme, not an outlier — gives  $3.2\times$ . Order and channel dominate; “batch” is not one number. The 019b H200 corner reaches  $\sim 860\times$  end-to-end at  $n = 10^4$  against its single-thread host mirror; the integrated 024 comparison is deliberately more conservative and compares the fastest eligible resident strategy on each platform at matched uniform Float64 cases. **Scope:** panels (a–c) time the far-field M2L stage alone (per route); panel (d)’s lifecycle series and panel (e)’s recurring step are complete evaluations including nearfield direct pairs, which are evaluated inside L2B. These scopes must not be multiplied together. **Caveats:** panel (c) is a scatter, not a curve — the route count is non-monotonic in  $P$  (`small_constp` runs 8, 59334, 144290, 31254, 144290, 15492 for  $P = 1, 2, 3, 4, 6, 8$ ), so the points carry no left-to-right ordering in  $P$ ; where two orders report the same route count the better speedup is drawn. It shows BLAS=1 only; the  $P = 4$  BLAS=64 route slice is in `fig02c_concat_speedup_routes.csv` and its  $P$ -axis view is panel (b). The three CPU configurations differ in  $n$  and tree depth (tiny  $n=256$ ,  $\ell=2$ ; small  $n=2048$ ,  $\ell=3$ ; medium  $n=20000$ ,  $\ell=4$ ) and only  $P \in \{2, 4\}$  was run for medium — gaps are genuine missing points, not zeros. Panel (d) points also differ in  $\ell$  ( $n=128$ :  $\ell=2$ ; 512: 3; 1000: 3;  $10^4$ : 4), and its host baseline is the single-thread host mirror of the same path, not the tuned legacy CPU FMM. The M2L-vs-host-recurrence point at  $n = 10^4$  was not recorded. Panel (e) compares different machines (m12-2-17 EPYC BLAS=1 and m13h-1-1 H200), but identical 024 case definitions and complete recurring-step timing; it is a platform comparison, not a kernel-only ratio.